

# Deep Learning for Cross-Sectional Stock Ranking

A sequence-to-rank pipeline with quant-aware evaluation

Runqi Wang

Shanghai Jiao Tong University  
Deep Learning and Applications, 2026 Spring

Final Project Presentation

noisy sequence modeling  $\rightarrow$  learning-to-rank  $\rightarrow$  portfolio construction.

# 1. Problem framing: not price prediction, but sequence-to-rank learning

## Machine-learning view

For stock  $i$  at date  $t$ , the input is a multivariate time-series window

$$X_{i,t} \in \mathbb{R}^{L \times d}, \quad d = 47,$$

and the neural network outputs one scalar score

$$s_{i,t} = f_{\theta}(X_{i,t}).$$

The score is used only for **ranking** stocks on the same date.

## Why this is hard for DL

- weak and noisy labels;
- temporal distribution shift;
- high-dimensional panel data;
- non-differentiable downstream decision: top- $k$  selection.

**Core idea:** convert a financial task into a supervised learning-to-rank problem with realistic out-of-sample evaluation.

## 2. Supervision: rank labels reduce non-stationary scale effects

### Cross-sectional rank target

Let  $r_{i,t+h}$  be the future  $h$ -day return. The label is

$$y_{i,t} = 2 \text{rankpct}_{j \in \mathcal{A}}(r_{j,t+h}) - 1 \in [-1, 1].$$

This converts raw return regression into a normalized daily ranking task.

### Signal diagnostic

$$\text{IC}_t = \text{Corr}_{i \in \mathcal{A}}(s_{i,t}, y_{i,t}).$$

IC is the cross-sectional correlation between model scores and future ranks.

### ML interpretation

- Rank labels act like date-wise normalization.
- The model learns relative strength, not absolute price levels.
- IC measures global ordering quality, analogous to a differentiable rank proxy.

### 3. Quant layer: from alpha score to active return

#### Alpha-score interpretation

The neural score is treated as a cross-sectional alpha signal:

$$\hat{\alpha}_{i,t} = \text{zscore}_{i \in \mathcal{A}}(s_{i,t}).$$

It is not a price forecast; it is a relative signal used to rank assets.

#### Portfolio map

$$w_t = \Pi_{\mathcal{C}}(\phi(\hat{\alpha}_t)),$$

where  $\phi$  is top- $k$  selection and  $\Pi_{\mathcal{C}}$  encodes long-only / weighting constraints.

#### Quant-aware utility

For benchmark weights  $b_t$ ,

$$R_{t+1}^{\text{act,net}} = w_t^\top r_{t+1} - b_t^\top r_{t+1} - \frac{c}{10000} \text{Turn}_t.$$

#### What this adds to ML

- IC evaluates signal quality.
- Active return evaluates benchmark-relative utility.
- Turnover penalizes unstable decisions.

## 4. Objective mismatch: differentiable training vs. top- $k$ evaluation

### Training is a surrogate problem

We optimize differentiable losses such as regression, correlation, and pairwise ranking losses:

$$\min_{\theta} \mathbb{E}_{(i,t)} \ell(f_{\theta}(X_{i,t}), y_{i,t}).$$

### Evaluation is a decision problem

The final portfolio is a thresholded decision rule:

$$S_t = \text{TopK}_{i \in \mathcal{A}}(s_{i,t}), \quad w_{i,t} = \frac{\mathbf{1}\{i \in S_t\}}{|S_t|}.$$

### Main technical difficulty

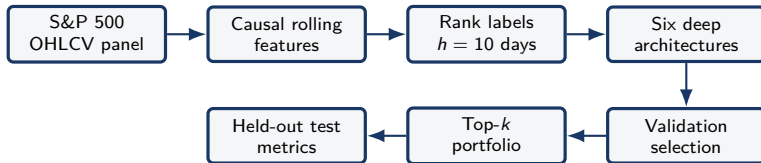
A model can improve average ranking correlation, but still fail if the improvement does not occur in the extreme upper tail selected by TopK.

### Transaction-cost layer

$$\text{Turn}_t = \frac{1}{2} \sum_i |w_{i,t} - w_{i,t-1}|,$$

$$R_t^{\text{net}} = R_t^{\text{gross}} - \frac{c}{10000} \text{Turn}_t.$$

## 5. Pipeline: a reproducible ML experimental protocol



### Computer-science controls

- chronological train/validation/test split;
- no future leakage in rolling features;
- model-wise hyperparameter search;
- three random seeds.

### Quant evaluation metrics

- IC / Rank signal: cross-sectional alpha quality;
- net and excess return: benchmark-relative utility;
- turnover: trading frequency and cost exposure;
- long-short spread: top-vs-bottom separation.

## 6. Model zoo as inductive-bias comparison

| Model    | Mathematical form                                                 | What it tests                                                     |
|----------|-------------------------------------------------------------------|-------------------------------------------------------------------|
| DLinear  | $s = w^\top \text{vec}(X) + b$                                    | Is a linear signal already enough?                                |
| ResMLP   | $h^{\ell+1} = h^\ell + g_\ell(h^\ell)$                            | Are nonlinear feature interactions useful?                        |
| GRU/LSTM | $h_\ell = \text{Gate}(x_\ell, h_{\ell-1})$                        | Does learned memory help under noisy temporal patterns?           |
| TCN      | $y_\ell = \sum_{m=0}^{K-1} W_m x_{\ell-dm}$                       | Are local temporal motifs captured by dilated convolution?        |
| PatchTST | $\text{Attn}(Q, K, V)$<br>$\text{softmax}(QK^\top / \sqrt{d_k})V$ | = Can patch attention model long-window interactions efficiently? |
| Ensemble | $s = \frac{1}{M} \sum_m \text{zscore}(s^{(m)})$                   | Does model averaging improve stable global ranking?               |

### Design principle

The experiment is not only “which model wins”, but which **inductive bias** best matches low-SNR financial sequences.

## 7. Why PatchTST-style tokenization is a natural fit

### Patch construction

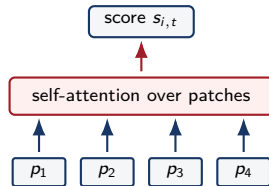
Split the length- $L$  window into patches  $p_k \in \mathbb{R}^{P \times d}$ , then project

$$u_k = W_p \text{vec}(p_k) + b_p.$$

The sequence length changes from  $L$  to approximately  $L/P$  tokens.

### Attention complexity

Day-level attention:  $O(L^2 d_k)$ . Patch-level attention:  $O((L/P)^2 d_k)$ .



### Intuition for non-experts

A patch is a short local pattern, like recent momentum or volatility. Attention then compares such patterns across the lookback window.



## 8. Loss design: robust regression + ranking regularization

### Robust base loss

$$\ell_{\delta}(s, y) = \begin{cases} \frac{1}{2}(s - y)^2 / \delta, & |s - y| \leq \delta, \\ |s - y| - \frac{1}{2}\delta, & |s - y| > \delta. \end{cases}$$

SmoothL1 is less sensitive to outlier labels than MSE.

### IC-aware loss

$$\mathcal{L}_{\text{IC}} = 1 - \text{Corr}_{i \in \mathcal{B}_t}(s_{i,t}, y_{i,t}).$$

### Pairwise ranking loss

$$\mathcal{L}_{\text{pair}} = \mathbb{E}_{(i,j)} \log(1 + \exp(-(s_i - s_j)\text{sign}(y_i - y_j))).$$

### Combined objective

$$\mathcal{L} = \mathcal{L}_{\text{SmoothL1}} + 0.1\mathcal{L}_{\text{IC}} + 0.05\mathcal{L}_{\text{pair}}.$$

This is a differentiable approximation to a non-differentiable ranking-and-selection pipeline.

## 9. Quant-aware model selection: validation is multi-objective

### Why validation cannot use loss alone

A lower supervised loss may still produce a poor strategy if it causes unstable ranks or weak top-tail separation. Therefore model selection uses a validation objective that combines signal and decision metrics.

### Schematic validation score

ValScore =

$$z(R^{\text{net}}) + z(R^{\text{excess}}) + z(\text{IC}) + z(R^{\text{LS}}) - \lambda z(\text{Turn}).$$

### Quant interpretation

This is a small, reproducible version of an alpha-research loop:

- 1 learn a signal;
- 2 convert it into holdings;
- 3 evaluate after costs;
- 4 select by validation performance, not test results.

**The ML model is judged by the utility of the decision rule it induces.**

## 10. Main empirical result: PatchTST wins the decision-level metric

| Model           | Test IC       | Long-only return | Excess return | Turnover      |
|-----------------|---------------|------------------|---------------|---------------|
| DLinear         | 0.0234        | 9.83%            | -7.42%        | 0.2218        |
| Ensemble        | <b>0.0512</b> | 18.25%           | 1.00%         | 0.2087        |
| GRU             | 0.0270        | 21.51%           | 4.25%         | <b>0.1057</b> |
| LSTM            | 0.0272        | 19.66%           | 2.40%         | 0.1201        |
| <b>PatchTST</b> | 0.0496        | <b>25.42%</b>    | <b>8.17%</b>  | 0.1945        |
| ResMLP          | 0.0395        | 17.22%           | -0.04%        | 0.3452        |
| TCN             | 0.0311        | 22.71%           | 5.46%         | 0.2364        |

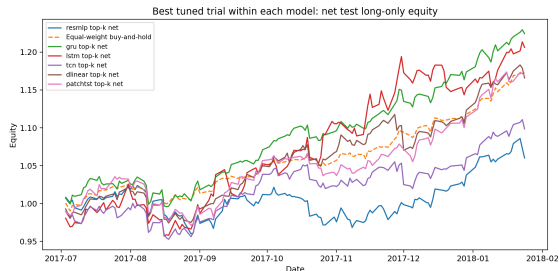
### Result

PatchTST achieves  $25.42\% \pm 4.68\%$  net long-only return and  $8.17\% \pm 4.68\%$  excess return over buy-and-hold.

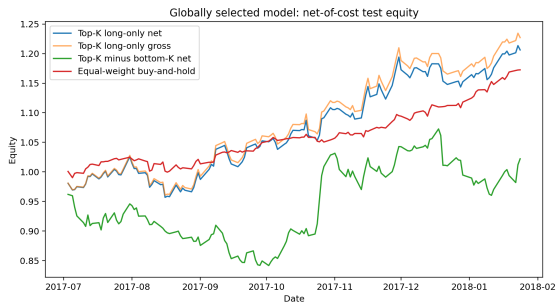
### Important nuance

The ensemble has the highest IC, but lower top- $k$  return. Better global correlation is not sufficient for better top-tail decisions.

# 11. Visual check: equity curves, not only table values



Model-family winners, seed 0

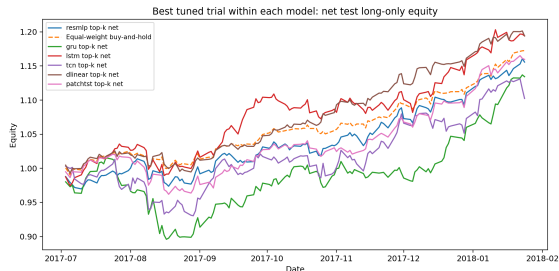


Global validation winner, seed 0

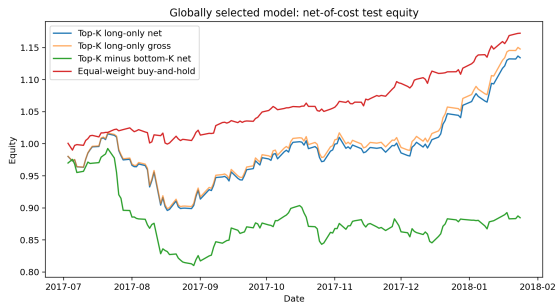
## What to look for

The equity curve checks whether performance is accumulated over time or caused by one short episode.  
The gross-net gap visualizes the transaction-cost penalty.

## 12. Visual check: equity curves, not only table values



Model-family winners, seed 1

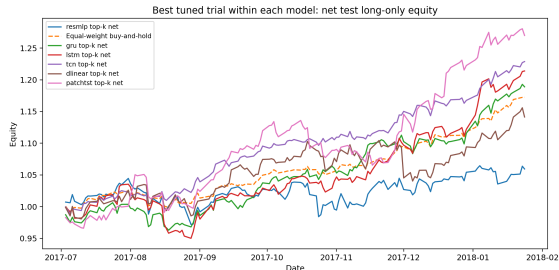


Global validation winner, seed 1

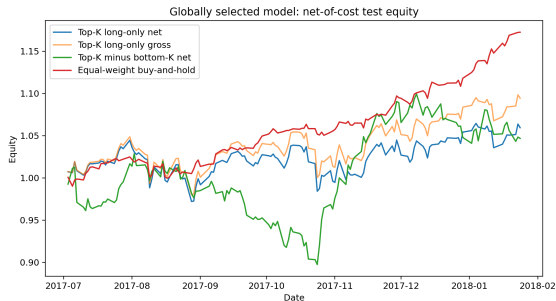
### What to look for

The equity curve checks whether performance is accumulated over time or caused by one short episode. The gross-net gap visualizes the transaction-cost penalty.

# 13. Visual check: equity curves, not only table values



Model-family winners, seed 2



Global validation winner, seed 2

## What to look for

The equity curve checks whether performance is accumulated over time or caused by one short episode. The gross-net gap visualizes the transaction-cost penalty.

## 14. Ablation I: global ranking improvement does not guarantee utility

| Loss                 | Test IC       | Long-only return | Excess return | Turnover      |
|----------------------|---------------|------------------|---------------|---------------|
| SmoothL1             | 0.0117        | <b>19.22%</b>    | <b>1.97%</b>  | <b>0.1289</b> |
| SmoothL1+IC          | 0.0149        | 17.44%           | 0.18%         | 0.2216        |
| SmoothL1+Pairwise    | <b>0.0326</b> | 18.46%           | 1.20%         | 0.3616        |
| SmoothL1+IC+Pairwise | 0.0203        | 15.57%           | -1.68%        | 0.1800        |

### ML lesson

Pairwise ranking improves IC from 0.0117 to 0.0326, so the surrogate objective does improve global ordering.

### Decision lesson

It also increases turnover and does not give the best net top- $k$  return. Surrogate alignment is incomplete.

## 15. Ablation II: post-processing is part of the algorithm

| Portfolio rule         | Long-only return | Excess return | Turnover      |
|------------------------|------------------|---------------|---------------|
| Equal, no smoothing    | 17.44%           | 0.18%         | 0.2216        |
| <b>Equal, EMA(0.7)</b> | <b>18.89%</b>    | <b>1.64%</b>  | <b>0.1541</b> |
| Inv-vol, no smoothing  | 17.85%           | 0.60%         | 0.2681        |
| Inv-vol, EMA(0.7)      | 18.83%           | 1.57%         | 0.1959        |

### Score smoothing as algorithmic stability

$$\bar{s}_{i,t} = \alpha s_{i,t} + (1 - \alpha) \bar{s}_{i,t-1}.$$

Smoother scores reduce rank switching, selected-set instability, and turnover.

### CS/ML takeaway

The deployment rule is not a detail after prediction; it is an algorithmic layer that changes generalization at the decision level.



## 16. Takeaways, limitations, and what is novel here

### Core contributions

- 1 A clean sequence-to-rank formulation for stock selection.
- 2 An inductive-bias comparison across linear, MLP, recurrent, convolutional, Transformer, and ensemble models.
- 3 Quant-aware evaluation connecting IC, active return, long-short spread, turnover, and transaction costs.

### Limitations

- public S&P 500 data may contain survivorship bias;
- only daily OHLCV features are used;
- this is an empirical course project, not a live trading system.

### Final message

The key deep-learning question is not whether a network can fit noisy returns, but whether its learned representation survives ranking, top- $k$  selection, temporal shift, and transaction costs.

# References

-  Gu, Kelly, and Xiu. Empirical Asset Pricing via Machine Learning. *Review of Financial Studies*, 2020.
-  Huber. Robust Estimation of a Location Parameter. *Annals of Mathematical Statistics*, 1964.
-  Hochreiter and Schmidhuber. Long Short-Term Memory. *Neural Computation*, 1997.
-  Cho et al. Learning Phrase Representations using RNN Encoder-Decoder. EMNLP, 2014.
-  Bai, Kolter, and Koltun. An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. 2018.
-  Vaswani et al. Attention Is All You Need. NeurIPS, 2017.
-  Zeng et al. Are Transformers Effective for Time Series Forecasting? AAAI, 2023.
-  Nie et al. A Time Series Is Worth 64 Words. ICLR, 2023.